

[14] ClickHouse: Lightning Fast Analytics for Everyone

저자: Robert Schulze, Tom Schreiber 외 다수 (ClickHouse Inc.) **학회/년도:** VLDB 2024 (Proceedings of the VLDB Endowment, Vol. 17, No. 12) **분량:** 약 14페이지 **역할군:** (C) 경쟁/비교 시스템

요약

ClickHouse는 **컬럼형 OLAP 엔진의 대표 시스템**으로, 대규모 데이터의 고속 분석에 특화되어 있다. 최근 벡터 검색 기능을 추가하여 범용 벡터 DB의 새로운 후보가 되고 있으며, Exqutor의 범용성을 보여주는 핵심 사례이다.

ClickHouse의 핵심 특징: 1. **벡터화 실행 엔진**: 한 번에 수천~수만 개 행을 배치 처리하여 CPU 캐시 효율성 극대화 2. **MergeTree 기반 컬럼형 저장**: 필요한 컬럼만 선택적으로 읽어 I/O 효율성 극대화 3. **스마트 데이터 스킵핑**: 각 데이터 블록별로 최소/최대/블룸필터 정보로 불필요한 블록 스킵 4. **다중 압축 알고리즘**: LZ4, ZSTD, Delta, Gorilla 등을 컬럼별로 자동 선택 5. **실시간 데이터 처리**: 초당 수백만 건의 데이터를 실시간으로 적재하면서 쿼리 실행

ClickHouse도 범용 DB에 벡터 검색을 추가한 경우로, **카디널리티 추정 문제를 안고 있을 것으로 예상된다**. Exqutor의 ECQO 개념을 ClickHouse에 적용하면 유사한 성능 향상을 기대할 수 있다.

상세분석

14.1 해결하고자 하는 문제: OLAP 엔진의 도전

기존 OLAP 엔진의 딜레마:

대부분의 데이터베이스는 다음 두 요구사항 중 하나에 특화된다:

1. **OLTP (Online Transactional Processing)**:
2. 소량의 데이터를 빠르게 읽고 쓰기 (예: 주문 삽입, 사용자 조회)
3. 트랜잭션 안전성, 동시성 제어 중심
4. 예: MySQL, PostgreSQL
5. **OLAP (Online Analytical Processing)**:

6. 대량의 데이터를 복잡하게 분석 (예: 월간 판매액 집계, 고객 세분화)
7. 쿼리 속도, I/O 효율성 중심
8. 예: 기존 Redshift, BigQuery

ClickHouse는 "두 마리 토끼를 모두 잡겠다"는 야망을 가지고 설계되었다:

ClickHouse의 목표: - **고속 적재:** 초당 수백만 건의 데이터를 스트리밍으로 받아서 적재 - **저지연 분석:** 분석 쿼리를 초 단위(또는 밀리초 단위)로 실행

예: "지난 1시간 동안 들어온 클릭 로그를 실시간 집계하면서, 동시에 사용자별 클릭 패턴을 분석하려면?"

14.2 핵심 기술 1: MergeTree 기반 컬럼형 저장

행 저장(Row-store) vs. 컬럼 저장(Column-store):

특성	행 저장	컬럼 저장
저장 순서	행 단위	컬럼 단위
접근 패턴	한 행의 모든 컬럼을 함께 읽음	필요한 컬럼만 선택적 읽음
적합한 워크로드	좁은 행 insert (OLTP)	넓은 행 집계 (OLAP)
압축률	낮음	높음 (같은 타입의 컬럼은 압축 잘됨)

MergeTree 구조:

INSERT (스트리밍) → 인메모리 버퍼 → (주기적) → 디스크 파트(Part) → (백그라운드) → 파트 병합

- **신규 파트(Recent Part):** 새로 삽입된 데이터는 작은 파트로 빨리 저장
- **병합(Merge):** 여러 작은 파트를 주기적으로 큰 파트로 병합하여 최적화
- **30+ 변형:**
 - **ReplacingMergeTree** : 키 기반으로 중복 제거하면서 병합
 - **SummingMergeTree** : 숫자 컬럼을 합산하면서 병합
 - **AggregatingMergeTree** : 사전 집계된 상태를 병합
 - **VersionedCollapsingMergeTree** : 버전 정보와 함께 행 삭제 지원

이 구조 덕분에 ClickHouse는 실시간 데이터 유입을 처리하면서도, 기존 데이터를 효율적으로 분석할 수 있다.

14.3 핵심 기술 2: 벡터화 쿼리 실행 (Vectorized Execution)

전통적 Volcano 모델의 한계:

기존 DB는 한 행씩 처리한다:

```
for each row:
    evaluate_condition(row)
    if matches:
        output(row)
```

이 방식의 문제: - CPU 캐시 미스 많음: 데이터가 스트리밍 방식으로 들어와서 캐시 지역성(locality) 낮음 - 분기 예측(Branch Prediction) 어려움: 조건 결과가 예측 불가능 - SIMD(Single Instruction Multiple Data) 활용 못함: 한 번에 한 행씩 처리하므로 벡터 명령어 활용 어려움

ClickHouse의 벡터화 실행:

```
BLOCK_SIZE = 8192행
for each block of rows:
    evaluate_condition_vectorized(block)
    if matches:
        output(block)
```

이렇게 하면: - **메모리 접근 패턴**: 같은 컬럼의 8192개 값이 연속으로 배치되므로 캐시에 잘 올라옴 - **분기 예측**: CPU가 분기 패턴을 학습할 여유가 생김 - **SIMD 활용**: AVX2, AVX-512 같은 벡터 명령어로 한 번에 32개 연산 수행

성능 향상: - CPU 캐시 효율: **3~5배** 향상 - SIMD 활용: **4~8배** 추가 향상 - 전체: **10~40배** 속도 향상

14.4 핵심 기술 3: 스마트 데이터 스킵핑 (Data Skipping)

그레인(Granule)과 스킵 인덱스:

ClickHouse는 데이터를 **그레인(granule)** 단위로 관리한다 (약 8,192행):

```
테이블 = [그레인1, 그레인2, 그레인3, ...]
```

각 그레인에 대해 **스킵 인덱스** 정보를 유지한다:

스킵 인덱스 유형	저장 정보	용도
MinMax	각 컬럼의 최소/최대 값	범위 쿼리 (WHERE date > '2024-01-01')
Bloom Filter	집합 멤버십 (이 그레인에 값 X가 있는가?)	정확한 값 조회 (WHERE user_id = 12345)
Set	특정 값 세트	카테고리 필터 (WHERE category IN ('A', 'B'))
Tuple	다중 컬럼 조합	복합 필터

쿼리 실행 시 스킵 인덱스 활용:

```
SELECT SUM(amount) FROM transactions
WHERE date >= '2024-01-01' AND date < '2024-02-01'
AND user_id = 12345;
```

1. MinMax 인덱스로 date 범위를 체크: 2024-01-01 ~ 2024-02-01 범위인 그래놀만 남음
2. Bloom 필터로 user_id = 12345를 체크: 해당 값이 없는 그래놀 제거
3. 남은 그래놀만 읽고 집계

이 과정에서 **대부분의 그래놀을 아예 읽지 않으므로**, I/O를 획기적으로 줄일 수 있다.

14.5 핵심 기술 4: 다중 압축 알고리즘

ClickHouse는 컬럼의 특성에 맞춰 압축 알고리즘을 선택할 수 있다:

LZ4 (빠른 압축): - 압축 속도: **매우 빠름** - 압축률: 낮음 (2~3배) - 사용: 자주 접근하는 컬럼, 실시간 쿼리

ZSTD (높은 압축률): - 압축 속도: 느림 - 압축률: 높음 (5~10배) - 사용: 자주 읽지 않는 아카이브 컬럼, 스토리지 절약

Delta (시계열 최적화): - 시계열 데이터의 연속된 값 차이만 저장 - 예: 타임스탬프 [1000000, 1000001, 1000002, ...] → 차이 [0, 1, 1, ...] → 매우 작은 값만 저장 - 압축률: **50~100배**

Gorilla (부동소수점 최적화): - Float/Double 값의 유사한 비트 패턴만 저장 - 예: 센서 데이터 온도 [23.456, 23.457, 23.458, ...] → 약간씩 변하는 마지막 비트만 저장 - 압축률: **10~50배**

개발자는 각 컬럼마다 최적의 압축 알고리즘을 지정할 수 있다:

```
CREATE TABLE metrics (
  timestamp DateTime CODEC(Delta, LZ4), -- 시간순 데이터: Delta + 추가압축
  temperature Float32 CODEC(Gorilla, ZSTD), -- 센서: 부동소수점 최적화 + 고압축
  value UInt64 CODEC(LZ4) -- 빠른 접근 필요
)
```

14.6 벡터 검색 기능 추가: 최근 발전

ClickHouse는 최근 벡터 검색 기능을 추가하고 있다:

- **annoy** 라이브러리: Spotify의 오픈소스 ANN 알고리즘
- **usearch** 라이브러리: 고성능 벡터 검색 라이브러리

하지만 아직 초기 단계로: - HNSW 인덱스 지원이 제한적 - 벡터 컬럼의 스킵 인덱스 지원 아직 미흡

14.7 성능 결과

ClickBench 벤치마크 (2024):

43개의 복잡한 OLAP 쿼리에 대해 모든 프로덕션급 DB를 상회하는 성능: - 100억 건 테이블에 대한 집계 쿼리: 초 단위 응답 (몇 초에서 수십 초) - 초당 수백만 건의 실시간 적재 처리 - 메모리 사용량: 경쟁사 대비 20~30%

14.8 본 논문과의 관계

ClickHouse는 범용 벡터 DB의 새로운 후보이다. ClickHouse의 옵티마이저도 벡터 카디널리티 추정 문제를 안고 있을 가능성이 높다:

- **MergeTree의 스킵 인덱스:** 고차원 벡터의 거리 분포를 정확히 표현할 수 없음
- **기존 히스토그램 기반 통계:** 벡터 거리의 비유클리드적 성질(고차원에서의 거리 집중 현상)을 포착하지 못함

Exqutor의 적용 가능성:

Exqutor의 ECQO 접근법(플래너 단계에서 실제 인덱스를 가볍게 탐색하여 카디널리티 추정)은 ClickHouse에도 개념적으로 적용 가능하다:

ClickHouse 플래너 → Exqutor 스타일의 카디널리티 추정 → 정확한 조인 순서 선택

이는 Exqutor의 범용성을 보여주며, 향후 ClickHouse에 벡터 검색이 더 성숙화되면 Exqutor의 개념을 적용할 수 있을 것으로 예상된다.

추가 제기 문제

1. **벡터 컬럼의 특수성:** ClickHouse의 벡터화 실행과 SIMD 최적화는 일반 컬럼에는 최적이지만, 벡터 컬럼은 다른 접근 패턴을 가진다. 벡터 검색은 순차 스캔이 아닌 무작위 접근(그래프 탐색)이므로, ClickHouse의 순차 스캔 최적화와 충돌할 수 있다.
2. **스킵 인덱스의 한계:** MergeTree의 스킵 인덱스는 범위 필터링에는 좋지만, 벡터 거리 임계값($\text{distance} < D$)을 정확히 표현하기 어렵다. 특히 고차원에서 거리 분포의 집중 현상이 스킵 인덱스의 효율성을 크게 떨어뜨린다.
3. **아직 초기 단계:** ClickHouse의 벡터 검색 기능은 아직 프로덕션 환경에 충분히 성숙되지 않았으므로, 대규모 벡터 데이터셋에 대한 성능 데이터가 부족하다.